2027考研重邮802数据结构模拟试卷
第1套 参考答案

对应第6版题面。程序填空接受等价写法；非唯一的树形、编码或路径按题内规则判分。算法实现仅给出一种正确方法，其他满足题目约束的解法同样有效。

评分建议用于核对作答要点。复杂度须与考生实际实现一致；不能仅因实现与参考代码不同而扣分。

一、程序阅读题

1. （5分）

填空：__1__＝15；__2__＝n-1；__3__＝n。

外层的 span 依次为 1、2、4、8（本次 n=16），内层执行次数之和为 1+2+4+8=15。一般为 1+2+…+2^(k-1)=2^k-1=n-1，所以时间为 O(n)。本次 outSum 还会打印 47：四轮所加读数之和分别为 3、4、9、31。

2. （5分）

填空：__1__＝prev->next；__2__＝p；__3__＝prev->next。

循环最终令 prev 指向 p 的前驱，包括 p 为首个数据结点时的头结点。先令 s->next=p，再令 prev->next=s。两次插入后的输出分别为 4->9 和 7->4->8->9。

3. （5分）

填空：__1__＝q->rear->next；__2__＝q->rear = NULL；__3__＝q->rear->next = x->next。

入队时新结点接在原队首之前、原队尾之后。出队时，若只剩一个结点则队列变空；否则把尾结点的 next 改为原队首的后继。示例先后输出 first=4, rear=9 与 second=9, empty=1, third-null=1。

4. （5分）

填空：__1__＝0；__2__＝1；__3__＝leafCount(t->left) + leafCount(t->right)。

空子树没有叶；叶结点自身贡献 1；非叶结点的叶数等于左右子树叶数之和。三次打印依次是 tree: 3、single: 1、empty: 0。

二、填空题

1.（3分）

第1空：a b + c * d -。先将 (a+b) 写成 a b +，再与 c 相乘，最后减 d。

2.（3分）

__1__＝1056。从 A[1,0] 到 A[3,2] 跨过 (3-1)×6+(2-0)=14 个元素，每个 4 字节，故地址为 1000+14×4=1056。

3.（3分）

填空：__1__＝4；__2__＝(r+1)%8 == f。

队长为 (r-f+8)%8=(2-6+8)%8=4。保留一个空槽时，再入队的尾位置将与 f 相等即为满。

4.（3分）

填空：__1__＝3；__2__＝1。

P[1..4]=abab 的最长相等真前后缀为 ab，长度 2，所以 next[5]=2+1=3。P[1..6]=ababac 没有非空相等真前后缀，所以 next[7]=0+1=1。

5.（3分）

__1__＝()。Tail(L)=((),c)；其第一个元素是空表 ()，而不是原子 c。

6.（3分）

__1__＝55。高度为 6 的完全二叉树前 5 层满，共 2^5-1=31 个结点。第 5 层有 16 个结点，其中 4 个是叶，即 12 个有第 6 层孩子；为使总数最大，让这 12 个结点各有两个孩子，得到 31+24=55。

7.（3分）

__1__＝9。第一次弹出 3 时，1、2、3 必须都已入栈，弹出 3 后栈内自底向上是 (1,2)，还可入栈 4、5。对剩下四次出栈按合法入弹规则枚举，得到 9 个不同序列。

8.（3分）

__1__＝7。按 AVL 插入并旋转后为根 8，左右孩子分别为 4、12，四个叶为 2、6、10、14。每个结点的左右子树高度相同，故七个结点的平衡因子全为 0。

9.（3分）

填空：__1__＝70；__2__＝3。

85 先放在下标 7，与下标 3 的 80 交换，上升到下标 3；其父结点 90 更大，停止。最终堆为 [90,70,85,20,50,60,80]，根的左孩子仍为 70。

10.（3分）

__1__＝7。满 6 层最多 2^6-1=63 个结点，容纳 95 个结点至少需要第 7 层；按题给中点规则生成的折半判定树最大深度为 7，最坏成功查找比较 7 次。

三、综合应用题

1．（10分）

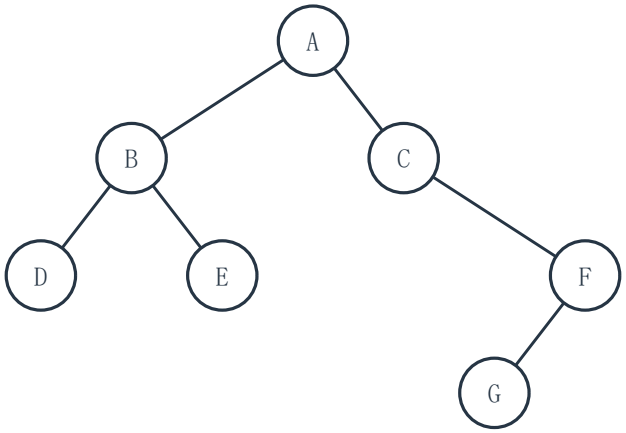
（1）重建结果（4分）：

树形见本题示意图。

等价孩子表：A:(B,C)， B:(D,E)， C:(NULL,F)， D:(NULL,NULL)， E:(NULL,NULL)， F:(G,NULL)，G:(NULL,NULL)。

（2）后序（2分）：DEBGFCA。

（3）指定线索（4分）：中序为 D B E A C G F。C 原左指针为空，故左指针指 A、左标志 1；C 原右指针指孩子 F、右标志 0。G 的左、右指针原本均为空，分别线索到前驱 C、后继 F；两侧标志都为 1。



重建二叉树

2. （10分）

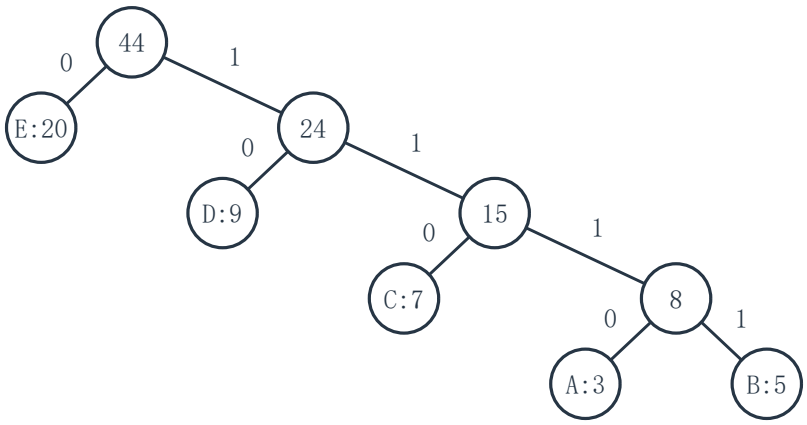
（1）合并次序（3分）：3(A)+5(B)=8；7(C)+8(AB)=15；9(D)+15(CAB)=24；20(E)+24=44。

（2）一组码字（5分）：

符号	A	B	C	D	E
码字	1110	1111	110	10	0

每次将较小权子树置左，左边标 0，右边标 1，符合题给并列规则。

（3）WPL（2分）：3×4+5×4+7×3+9×2+20×1=91；亦等于合并新权之和 8+15+24+44=91。



一组Huffman树（叶结点为字符:频率，左0右1）

3. （10分）

（1）散列表（4分）：

地址	0	1	2	3	4	5	6	7	8	9	10
键	-	-	-	14	25	36	5	16	-	-	-

（2）成功 ASL（3分）：按插入顺序，五键的探测次数依次为 1,2,3,2,3；ASL成功=(1+2+3+2+3)/5=11/5=2.2。

（3）失败 ASL（3分）：起始地址 0～10 的探测次数依次为 1,1,1,6,5,4,3,2,1,1,1，所以 ASL失败=26/11≈2.364。失败时最后遇到的空槽也算一次探测。

4. （10分）

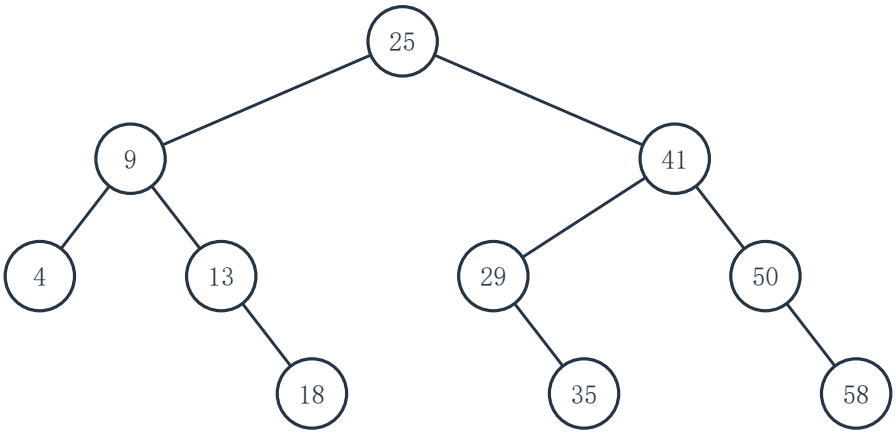
（1）折半判定树（5分）：

树形见本题示意图。

等价孩子表：25:(9,41)， 9:(4,13)， 13:(NULL,18)， 41:(29,50)， 29:(NULL,35)， 50:(NULL,58)；4、18、35、58 为叶。

（2）成功 ASL（3分）：比较次数为 1 的键 1 个、为 2 的 2 个、为 3 的 4 个、为 4 的 3 个，所以 ASL成功=(1×1+2×2+3×4+4×3)/10=29/10=2.9。

（3）失败查找 27（2分）：依次比较 25→41→29，共 3 次；随后左侧区间变空。



折半查找判定树

5. （10分）

（1）Prim（5分）：从 A 出发按入树顺序选 AB(2)→BC(3)→CD(4)→DE(6)→EF(1)。

（2）Kruskal（3分）：按全图边权由小到大考虑，纳入 EF(1)→AB(2)→BC(3)→CD(4)→DE(6)；权值 5 的 AC 会成环而跳过。

（3）总权（2分）：1+2+3+4+6=16。两种算法选到同一棵树，加入顺序不同。

6. （10分）

（1）普通 BST 与比较次数（3分）：

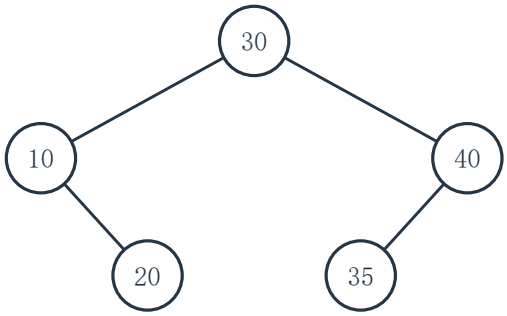
树形见本题示意图。

键 30,10,20,40,35 的成功比较次数依次为 1,2,3,2,3。

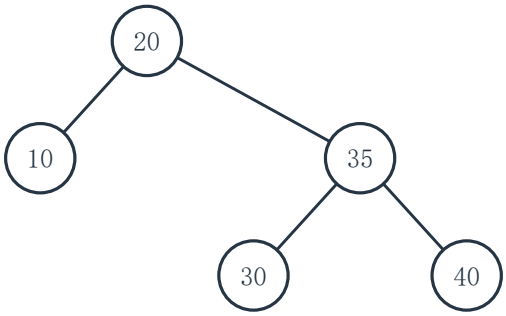
（2）普通 BST 成功 ASL（2分）：(1+2+3+2+3)/5=11/5=2.2。

（3）AVL（5分）：插入 20 后，结点 30 首次出现 **LR** 失衡，先对 10 左旋、再对 30 右旋；插入 35 后，结点 30 出现 **RL** 失衡，先对 40 右旋、再对 30 左旋。最终 AVL 为：

树形见本题示意图。



普通BST



最终AVL树

7. （10分）

（1）首枢轴快排划分（5分）：结果数组为 [22,17,31,28,9,42,54,63]，枢轴 42 最终下标为 **5**（0 基）。按题给代码，右扫依次把 22、31 填到左侧空位，左扫依次把 63、54 填到右侧空位，最终在下标 5 放回 42。

（2）自下而上建小根堆（5分）：结果数组为 [9,17,31,22,42,54,63,28]。按0基下标从最后非叶结点 3、2、1、0 依次下滤后，四个状态依次是：

3: [42,17,63,22,9,54,31,28]
2: [42,17,31,22,9,54,63,28]
1: [42,9,31,22,17,54,63,28]
0: [9,17,31,22,42,54,63,28]

四、算法设计与分析题

1．（8分）

基本思想：同时扫描有序的 A、B。A 当前键较小时将它保留并前进；B 当前键较小时只前进 B；两键相等时从 A 摘下并释放该结点。保留一个 A 的前驱指针，删除首个数据结点时也能正确接回头结点。

C 语言实现：

```
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *next;
} Node;

void listDifference(Node *A, const Node *B) {
    Node *prev = A, *pa = A->next;
    const Node *pb = B->next;
    while (pa != NULL && pb != NULL) {
        if (pa->data < pb->data) {
            prev = pa;
            pa = pa->next;
        } else if (pa->data > pb->data) {
            pb = pb->next;
        } else {
            Node *old = pa;
            pa = pa->next;
            prev->next = pa;
            free(old);
            pb = pb->next;
        }
    }
}
```

复杂度：设 A、B 分别有 m、n 个数据结点。两个游标都只向前，时间 O(m+n)，额外指针空间 O(1)；释放原结点不算新增辅助空间。A 或 B 为空时循环直接结束。**评分建议：**双指针思路 1 分；接链、释放与边界正确的代码 5 分；时间及辅助空间各 1 分，共 8 分。

2. （10分）

基本思想：先遍历所有根叶路径，求最大的路径和；再遍历一次，沿当前路径把结点值暂写入调用者提供的 path。第一次遇到和等于最优值的叶结点时固定 pathLen 并返回。这样即使结点值为负也无需用 0 代替不存在的子树。

C 语言实现：

```
typedef struct TNode {
    int data;
    struct TNode *left, *right;
} TNode;

static void scanBest(const TNode *p, long long sum,
                     long long *best, int *haveBest) {
    sum += p->data;
    if (p->left == 0 && p->right == 0) {
        if (!*haveBest || sum > *best) *best = sum;
        *haveBest = 1;
        return;
    }
    if (p->left != 0) scanBest(p->left, sum, best, haveBest);
    if (p->right != 0) scanBest(p->right, sum, best, haveBest);
}

static int savePath(const TNode *p, long long sum, long long best,
                    int path[], int depth, int *pathLen) {
    if (p == 0) return 0;
    path[depth] = p->data;
    sum += p->data;
    if (p->left == 0 && p->right == 0) {
        if (sum == best) {
            *pathLen = depth + 1;
            return 1;
        }
        return 0;
    }
    if (savePath(p->left, sum, best, path, depth + 1, pathLen)) return 1;
    return savePath(p->right, sum, best, path, depth + 1, pathLen);
}

int maxRootLeafSum(const TNode *root, int *path, int *pathLen) {
    long long best = 0;
    int haveBest = 0;
    *pathLen = 0;
    scanBest(root, 0, &best, &haveBest);  /* 题设 root 非空 */
    savePath(root, 0, best, path, 0, pathLen);
    return (int)best;                     /* 题设保证路径和在int范围内 */
}
```

复杂度：两次遍历各至多访问每个结点一次，时间 O(n)；递归调用栈深度 O(h)，最坏 O(n)。path 为题给输出缓冲区，至多写 h 个结点，不另复制整条路径；若计入输出占用则再计 O(h)。**评分建议：**负值与根叶路径的思路 2 分；两趟遍历、路径保存和边界代码 6 分；时间、空间各 1 分，共 10 分。

3. （12分）

基本思想：一次 DFS 从 s 出发，把 w 视为禁止访问。每个可访问顶点只标记一次；递归当前路径直接写入 path[depth]。到 t 时固定长度并立即返回。某分支失败后，已完全探索过的顶点仍保持已访问；它不可能在别的分支再通向 t，因此既避免重复顶点，也保证线性复杂度。

C 语言实现：

```
typedef struct Arc {
    int to;
    struct Arc *next;
} Arc;
typedef struct {
    int V, E;
    Arc **head;
} Graph;

static int dfsAvoid(const Graph *G, int u, int t, int w,
                    unsigned char seen[100], int path[],
                    int depth, int *pathLen) {
    seen[u] = 1;
    path[depth] = u;
    if (u == t) {
        *pathLen = depth + 1;
        return 1;
    }
    for (const Arc *e = G->head[u]; e != 0; e = e->next) {
        int v = e->to;
        if (v != w && !seen[v] &&
            dfsAvoid(G, v, t, w, seen, path, depth + 1, pathLen))
            return 1;
    }
    return 0;
}

int pathAvoidingVertex(const Graph *G, int s, int t, int w,
                       int *path, int *pathLen) {
    unsigned char seen[100] = {0};
    *pathLen = 0;
    return dfsAvoid(G, s, t, w, seen, path, 0, pathLen);
}
```

复杂度：每个顶点最多进入 DFS 一次，每条无向边的两条邻接记录最多扫描一次，时间 O(V+E)；seen 与递归栈共 O(V) 辅助空间。输出 path 容量 V 由调用者提供，实际写入至多 V 个编号。**评分建议：**禁点与单次DFS思路 2 分；标记、路径保存、成功/失败边界的代码 8 分；时间与空间复杂度各 1 分，共 12 分。

2027考研重邮802数据结构模拟试卷
第2套 参考答案

对应第6版题面。程序填空接受等价写法；非唯一的树形、编码或路径按题内规则判分。算法实现仅给出一种正确方法，其他满足题目约束的解法同样有效。

评分建议用于核对作答要点。复杂度须与考生实际实现一致；不能仅因实现与参考代码不同而扣分。

一、程序阅读题

1. （5分）

__1__ = 7；__2__ = 2M(n-1)+1；__3__ = 2^n。

两次规模为 n-1 的递归和一次 move 给出递推式。由 M(1)=1 得 M(n)=2^n-1，因此 n=3 时执行 7 次。每次移动本身为 O(1)，总时间 O(2^n)；递归栈深度为 n。

2. （5分）

__1__ = q；__2__ = y；__3__ = q。

交换后局部链为 x<->p<->q<->y：p->next 指向 q，q->next 指向 y，且 y 的前驱变为 q；其余三条指针由原代码更新。示例前向输出从 10 20 变为 20 10，tail.prior 为 10。

3. （5分）

__1__ = s->topL + 1 == s->topR；__2__ = --s->topR；__3__ = s->topR++。

左栈顶右边紧邻右栈顶即满。右栈入栈先向左移动 topR，再写数组；右栈出栈先取原 topR 处元素，再向右移动。示例出栈值为 8，最后 topL=0, topR=5。

4. （5分）

__1__ = 0；__2__ 可填 a - b > 1 || b - a > 1；__3__ = 1 + (a > b ? a : b)。

空树高度为 0；任一子树返回 -1 表示不平衡，随后检查左右高度差是否超过 1。若平衡，当前高度为较大子树高度加 1。示例输出 1 0 1。

二、填空题

1. （3分）

__1__ = 14。依次算 8-3=5、5×2=10、10+4=14；弹栈时先取的是右操作数。

2. （3分）

__1__ = 14。四个不同元素依次入栈的合法出栈序列数是第 4 个 Catalan 数：C_4 = (1/5)·C(8,4)=14。

3. （3分）

__1__ = 3；__2__ = 2。队长 (rear-front+8) mod 8=(1-6+8) mod 8=3；入队后 rear=(1+1) mod 8=2。

4. （3分）

__1__ = 5；__2__ = 0。P[1..5]=aaaaa 的最长真前后缀为 aaaa，故 next[6]=4+1=5；整个 aaaaab 的末字符为 b，非空前缀均以 a 开始，故 π(6)=0。

5. （3分）

__1__ = 468。从首元素到 A[2,3,2] 的元素偏移量为 (2-1)·4·3+(3-2)·3+(2-0)=17，字节偏移量为 17×4=68，地址为 400+68。

6. （3分）

__1__ = 3。森林中每棵树都有“边数=结点数-1”，总边数为 12-树数=9，故有 3 棵树。

7. （3分）

__1__ = n+1。共有 2n 个孩子指针；无共享结点的二叉树有 n-1 条孩子边，因此空指针数为 2n-(n-1)=n+1。

8. （3分）

__1__ = 5。以题定下取整中点划分，16 个关键字的成功判定树最深结点比较 5 次；例如最右端关键字依次经过下标 7、11、13、14、15。

9. （3分）

__1__ = 3；__2__ = 2。12 先放下标7，与下标3的15比较并交换；再与下标1的10比较后停止，所以最终在下标3，共比较2次。

10. （3分）

__1__ = 8。严格二叉树设叶数 L、双孩子结点数 I，则 L=I+1，L+I=15，解得 L=8。

三、综合应用题

1．（10分）

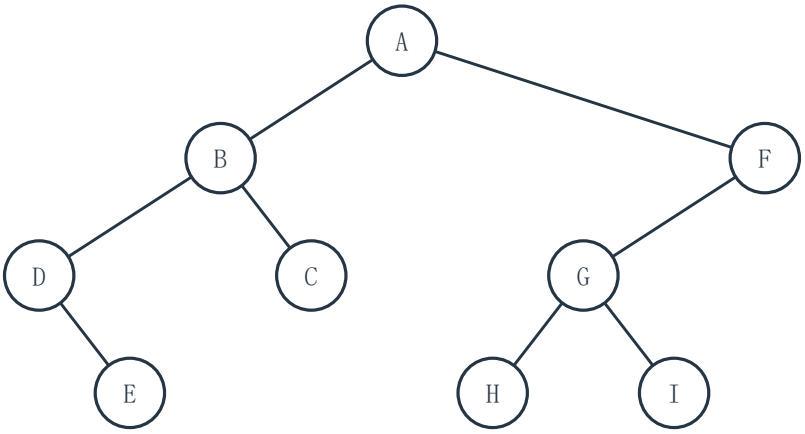
（1）按左孩子、右孩子次序列出：

结点	A	B	C	D	E	F	G	H	I
左孩子	B	D	NULL	NULL	NULL	G	H	NULL	NULL
右孩子	F	C	NULL	E	NULL	NULL	I	NULL	NULL

（2）原森林先根遍历为 **A B D E C F G H I**；后根遍历为 **D E B C A H G I F**。

（3）根为 A、F，共 **2** 棵树；边数为 **7**。

先根遍历每棵树时先访问根再自左至右递归孩子；后根遍历先访问孩子再访问根。转换中的 A→F 是根之间的右兄弟关系，不是森林的一条父子边。



转换后的孩子兄弟二叉树

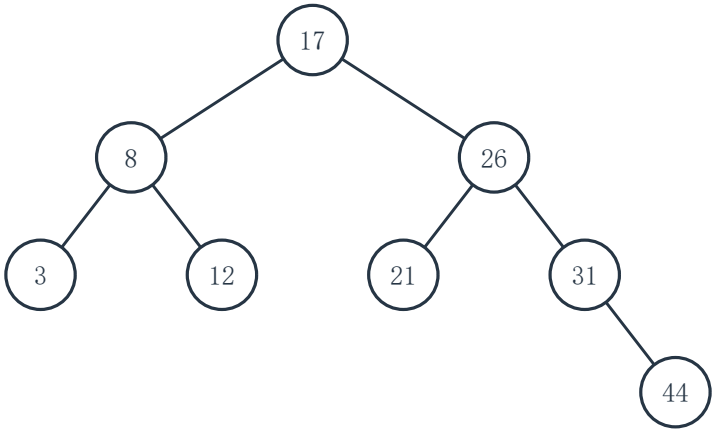
2. （10分）

（1）成功判定树（括号内为左右孩子）如下：

17:(8,26)
 8:(3,12)　　26:(21,31)
 3:(NULL,NULL)　　12:(NULL,NULL)
21:(NULL,NULL)　　31:(NULL,44)
44:(NULL,NULL)

（2）按比较次数分组：第 1 层 1 个，第 2 层 2 个，第 3 层 4 个，第 4 层 1 个。成功 ASL=(1×1+2×2+3×4+4×1)/8=21/8。

（3）九个失败间隙自左至右的比较次数为 3，3，3，3，3，3，3，4，4。失败 ASL=(7×3+2×4)/9=29/9。



折半查找判定树

3．（10分）

（1）线性探测表为：

槽号	0	1	2	3	4	5	6	7	8	9	10
关键字	空	12	23	34	4	15	26	7	空	空	空

拉链法的非空桶：1:12→23→34，4:4→15→26，7:7。

（2）线性探测按题目给定七个关键字的成功比较次数为 1,2,3,1,2,3,1，成功 ASL=13/7。拉链法对应链内位置也为 1,2,3,1,2,3,1，成功 ASL=13/7。

（3）线性探测按起始地址 0～10 的失败比较次数为 1,8,7,6,5,4,3,2,1,1,1，失败 ASL=39/11。拉链法按桶号 0～10 的失败比较次数为 0,3,0,0,3,0,0,1,0,0,0，失败 ASL=7/11。线性探测把首个空槽计入，拉链法空桶不计比较。

4．（10分）

（1）逐轮选入顶点并松弛后的暂定距离如下：

选入	B	C	D	E	F
A	0	4	∞	∞	∞
B	0	2	7	∞	∞
C	0	2	3	8	∞
D	0	2	3	6	∞
E	0	2	3	6	∞

因此选入顺序为 **A，B，C，D，E**；F 不可达，不属于题目要求列出的轮次。

（2）最终距离：d(A)=0，d(B)=0，d(C)=2，d(D)=3，d(E)=6，d(F)=∞。

（3）一条 A 到 E 的最短路径为 **A→B→C→D→E**，长度 0+2+1+3=6。

5．（10分）

（1）按事件 0～5 顺序，最早发生时间 ve=(0,3,2,6,6,8)。例如事件3取 max(3+2,2+4)=6，事件5取 max(3+3,6+2,6+1)=8。

（2）最迟发生时间 vl=(0,4,2,6,7,8)。从汇点5的8逆推：vl(3)=8-2=6，vl(4)=8-1=7，vl(1)=min(6-2,7-3,8-3)=4。

（3）工期 8。活动 B（0→2）、E（2→3）、G（3→5）的最早开始时间等于最迟开始时间，故关键活动为 B、E、G；全部关键路径仅 0→2→3→5。

6. （10分）

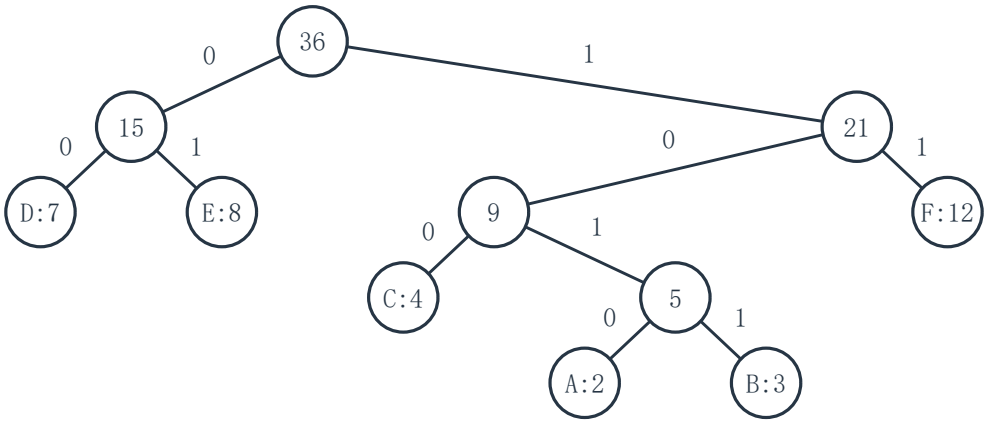
（1）合并顺序：2+3=5，4+5=9，7+8=15，9+12=21，15+21=36。

（2）一组符合“小权值左0、大权值右1”的码字为：

符号	A	B	C	D	E	F
码字	1010	1011	100	00	01	11

对应树的根左右子树权值分别为15（由 D7、E8 构成）和21（由9、F12构成）；权值9又由 C4 与 (A2,B3) 权值5 构成。

（3）WPL=2×4+3×4+4×3+7×2+8×2+12×2=86，也等于五次合并权值之和 5+9+15+21+36=86。



一组Huffman树（叶结点为字符:频率，左0右1）

7．（10分）

（1）甲完成 i=4 后：[17,24,36,53,72,91,45,68]。

（2）甲完成 i=6 后：[17,24,36,45,53,72,91,68]。i=5 处理91不移动，i=6 将45插到36与53之间。

（3）乙第一趟挖坑划分后：[45,17,36,24,53,91,72,68]。

（4）枢轴53最后处于下标 4。从右侧先搬45到左端，再从左侧搬72到右端；继续从右侧搬36到左侧，两个指针在下标4相遇，填入53。

四、算法设计与分析题

1．（8分）

基本思想。遍历一次原链，取下当前数据结点，按数据是否小于0依次接到“负数链”或“非负数链”的尾部；两条链内部始终按原扫描顺序追加。最后把负数链接到非负数链前，并从原头结点接入。两个临时哨兵是局部变量，不新建数据结点。

```
#include <stddef.h>
typedef struct Node {
    int data;
    struct Node *next;
} Node;

void stableNegFirst(Node *L) {
    Node neg = {0, NULL}, nonneg = {0, NULL};
    Node *tn = &neg, *tp = &nonneg;
    Node *p = L->next;
    while (p != NULL) {
        Node *next = p->next;
        p->next = NULL;
        if (p->data < 0) {
            tn->next = p;
            tn = p;
        } else {
            tp->next = p;
            tp = p;
        }
        p = next;
    }
    tn->next = nonneg.next;
    L->next = neg.next;
}
```

复杂度。n 为数据结点数，每个结点只访问一次，时间 O(n)，辅助空间 O(1)。空表或无负数时 tn 仍指向局部哨兵，连接后 L->next 正确；无非负数时末尾接 NULL。

评分建议（8分）：稳定分组思想1分；正确拆链、尾插和最终连接5分；时间复杂度1分；辅助空间复杂度1分。未保持两组各自原序的实现不能取得稳定连接相关分。

2. （10分）

基本思想。后序遍历，每个结点先取得左右子树的叶数；若左右孩子都为空，写入1，否则写入左右结果之和。递归返回值就是当前子树叶数，因此每个结点只处理一次，孩子指针保持原值。

```
#include <stddef.h>
typedef struct TNode {
    int data;
    int count;
    struct TNode *left, *right;
} TNode;

static int leafCount(TNode *p) {
    if (p == NULL) return 0;
    if (p->left == NULL && p->right == NULL) {
        p->count = 1;
        return 1;
    }
    int a = leafCount(p->left);
    int b = leafCount(p->right);
    p->count = a + b;
    return p->count;
}

void fillLeafCounts(TNode *root) {
    (void)leafCount(root);
}
```

复杂度。n 为结点数、h 为树高，时间 O(n)，辅助空间 O(h)（递归调用栈；空树 O(1)，链状树最坏 O(n)）。

评分建议（10分）：后序汇总思想2分；完整可执行代码6分，其中空树/叶结点及内部结点写值均应正确；时间复杂度1分；计入递归栈的空间复杂度1分。

3. （12分）

基本思想。无向图中从 s 到 t 存在边数不超过 k 的路径，当且仅当最短路边数不超过 k。用 BFS 求最短路和前驱；BFS 第一次到达 t 时得到一条最短路，其 BFS 树路径天然不重复顶点。若 s=t，路径只有 s；若 k<0，按题意直接失败。逆着前驱回溯后反向写入 path。

```
#include <stddef.h>
typedef struct Arc {
    int to;
    struct Arc *next;
} Arc;
typedef struct {
    int V, E;
    Arc **head;
} Graph;

int pathAtMostK(const Graph *G, int s, int t, int k,
                int *path, int *pathLen) {
    *pathLen = 0;
    if (k < 0) return 0;
    if (s == t) {
        path[0] = s;
        *pathLen = 1;
        return 1;
    }

    int dist[100], parent[100], queue[100], rev[100];
    for (int i = 0; i < G->V; ++i) {
        dist[i] = -1;
        parent[i] = -1;
    }
    int front = 0, rear = 0;
    dist[s] = 0;
    queue[rear++] = s;
    while (front < rear && dist[t] == -1) {
        int u = queue[front++];
        for (const Arc *a = G->head[u]; a != NULL; a = a->next) {
            int v = a->to;
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            parent[v] = u;
            queue[rear++] = v;
        }
    }
    if (dist[t] == -1 || dist[t] > k) return 0;

    int len = 0;
    for (int v = t; v != -1; v = parent[v]) rev[len++] = v;
    for (int i = 0; i < len; ++i) path[i] = rev[len - 1 - i];
    *pathLen = len;
    return 1;
}
```

复杂度。BFS 至多扫描 V 个顶点及邻接表中 2E 条弧，时间 O(V+E)；dist、parent、队列与倒序路径数组合计 O(V) 辅助空间，输出数组由调用者提供。若 s=t 或 k<0，提前返回为 O(1)。

评分建议（12分）：把存在性化为 BFS 最短路并处理边界2分；初始化、访问与前驱记录5分；限长判断及无重复路径回溯3分；时间与空间复杂度各1分。仅判断可达而未输出合法路径不能取得回溯部分分。

2027考研重邮802数据结构模拟试卷
第3套 参考答案

对应第6版题面。程序填空接受等价写法；非唯一的树形、编码或路径按题内规则判分。算法实现仅给出一种正确方法，其他满足题目约束的解法同样有效。

评分建议用于核对作答要点。复杂度须与考生实际实现一致；不能仅因实现与参考代码不同而扣分。

一、程序阅读题

1. （5分）

各空依次为：1、2、4、8；32；k+1；O(n log n)。

内层每轮执行 8 次，外层对样例执行 4 轮。一般 n=2^k 时，外层取值从 2^0 到 2^k，共 k+1 轮，故操作次数为 n(k+1)。

2. （5分）

__1__ = head->next；__2__ = p->next；__3__ = q->key < best->key。

打印：(1,-) (2,-) (4,b) (4,a)；最坏时间复杂度：O(n^2)。首轮把最后的 (1,-) 与 (4,a) 整记录互换，因此原来后面的 (4,b) 排在 (4,a) 前；选第一个同键最小结点也不能保证此交换式选择排序稳定。

3. （5分）

__1__ = q->rear->next = p；__2__ = q->rear = q->front；__3__ = q->front == q->rear。

打印：4 7；单次 enq 或 deq 均为 O(1)。头结点始终保留；删掉最后一个数据结点时必须把 rear 复位到头结点。

4. （5分）

__1__ = g[u][v] && !seen[v]；__2__ = dfs(v)；__3__ = !seen[u]；时间复杂度 O(V^2)。

每个顶点只进入 dfs 一次，但每次都要扫描邻接矩阵的一整行，共 V 行；外层找各连通分量根的开销被 V^2 包含。样例会识别 {0,1,2} 与 {3} 两个分量。

二、填空题

1. （3分）

依次填 2、0、5。四次入队后 rear=4；两次出队后 front=2、count=2；再入三次使 rear=(4+3) mod 7=0、count=5。

2. （3分）

填 27。严格二叉树中叶数为内部结点数加1；设叶数 L，则总数为 2L-1=53。

3. （3分）

填 14。首次出栈4意味着此前必须按序压入1、2、3、4并立即弹出4；其后对栈中3、2、1及待入的5、6按合法入出栈次序分支枚举，得到14个完整序列。

4. （3分）

填 0,1,1,2,3,1,2。对 j=1..7，前 j-1 字符的最长真前后缀长度依次为“不适用、0、0、1、2、0、1”，按给定约定加1（首项固定0）。

5. （3分）

填 16。后缀式先算 3 2 + = 5，得 8×5=40；再算 9 5 - = 4，得 40÷4=10，最后 10+6=16。

6. （3分）

填 12。出边数为 4×3+3×2+2×1=20，总结点数21；已知非叶结点9个，所以叶结点12个。

7. （3分）

填 n。每个顶点至少有一条出弧，故弧数至少 n；一个覆盖全部 n 个顶点的有向环恰用 n 条弧并强连通。

8. （3分）

填 23。每棵树的边数等于其结点数减1，因此森林总边数为 27-4=23。

9. （3分）

填 (0,4,1,9,7,6,2)。新键0先占7号位，与3号位的2交换，再与1号位的1交换。

10. （3分）

填 24。二叉链表共有 2n 个孩子指针，其中非空指针恰为树边数 n-1，空指针数为 2n-(n-1)=n+1=24。

三、综合应用题

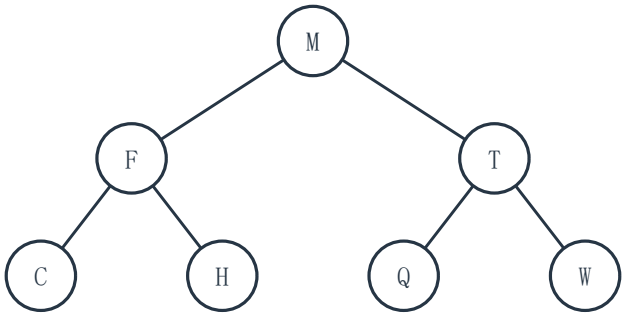
1．（10分）

（1）原树如下；横向标注分别是左、右孩子：

树形见本题示意图。

（2）后序：C,H,F,Q,W,T,M。

（3）中序是 C,F,H,M,Q,T,W，所以 C 的空左指针→NULL；H 的空右指针→M；Q 的空左指针→M；W 的空右指针→NULL。题中仅询问原本为空的指针，实际孩子指针仍保持原孩子。



重建二叉树

2. （10分）

（1）普通 BST：

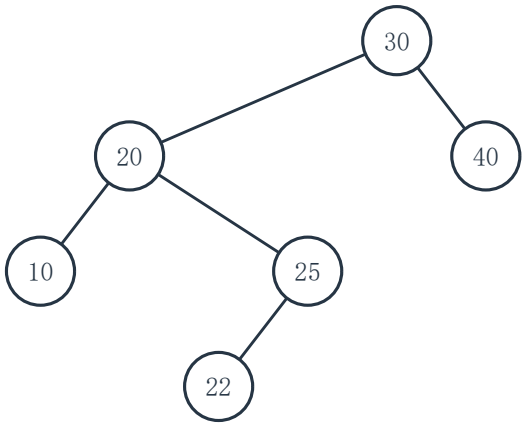
树形见本题示意图。

（2）插入22后，向上回溯首次失衡的结点是 30，属于 **LR** 型：先对其左孩子20左旋，再对30右旋。

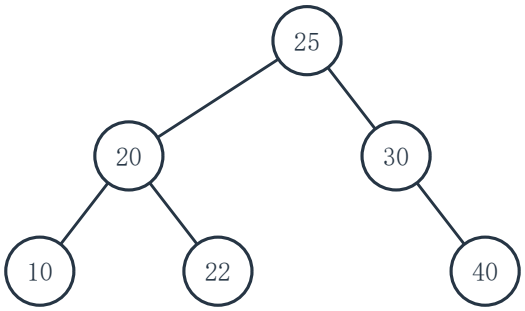
（3）AVL 调整后：

树形见本题示意图。

（4）普通 BST 查找22：30→20→25→22，比较4次；AVL 查找22：25→20→22，比较3次。



普通BST



最终AVL树

3. （10分）

（1）按百分数权值写，七次合并为：2+3=5，5+6=11，7+10=17，11+17=28，19+21=40，28+32=60，40+60=100。新权值依次为 5、11、17、28、40、60、100（百分数）。

（2）WPL 等于各次新权值之和，(5+11+17+28+40+60+100)/100 = 2.61 bit/字符。

（3）3位定长编码平均 3 bit/字符；比 Huffman 多 3-2.61=0.39 bit/字符。

（4）8个叶、每个内部结点有两个孩子，内部结点7个，总结点数 15。

4．（10分）

（1）插入后槽位：

槽号	0	1	2	3	4	5	6	7	8	9	10
键	22	33	空	14	25	36	47	空	空	空	空

（2）按 22,33,14,25,36,47 的成功探测次数为 1,2,1,2,3,4，所以 ASL成功=13/6≈2.167。

（3）初址 0..10 的失败探测次数依次为 3,2,1,5,4,3,2,1,1,1,1，故 ASL失败=24/11≈2.182。例如从3出发连续查3、4、5、6、7，遇7号空槽才停，计5次。

5. （10分）

（1）Kruskal 按表序检查到选满4条边：

检查边	AB	AC	BC	BD	CD	DE
结果	入树	入树	因成环略过	入树	因成环略过	入树

CE 尚未轮到，不必检查。

（2）最小生成树边集 {AB,AC,BD,DE}，总权值 1+1+2+3=7。

（3）Prim 从 A 起，等权按表序，入树次序：AB→AC→BD→DE。

6. （10分）

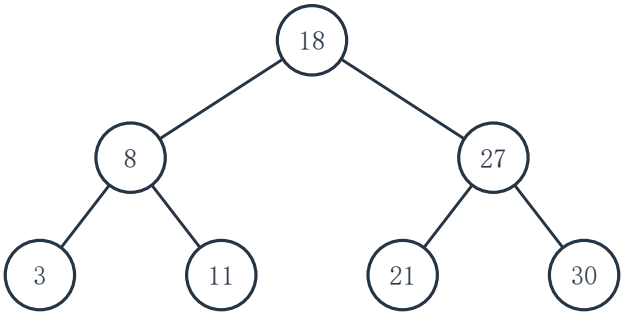
（1）折半判定树：

树形见本题示意图。

（2）各层结点数1、2、4，故 ASL成功=(1×1+2×2+4×3)/7=17/7≈2.429。

（3）查找21的比较顺序：18→27→21。

（4）查找25时比较 18→27→21，然后进入空区间，故比较 3 次。



折半查找判定树

7. （10分）

（1）第一趟（长度1段两两归并）：(3,8a,1,7,2,8b,4,6)。

（2）第二趟（长度2段两两归并）：(1,3,7,8a,2,4,6,8b)。

（3）第三趟（长度4段归并）：(1,2,3,4,6,7,8a,8b)。两个键8相等时先取左段的8a，故保持稳定次序。

四、算法设计与分析题

1．（8分）

基本思想。 三个游标分别指向三条未处理链表的首结点。每次取非空游标中键最小的原结点，摘下并接到结果尾部。键相同时任选一条；最后把结果尾结点的 next 设为 NULL。

```
#include <stddef.h>
typedef struct Node { int key; struct Node *next; } Node;

Node *mergeThree(Node *a, Node *b, Node *c) {
    Node *head = NULL;
    Node **tailNext = &head;
    while (a != NULL || b != NULL || c != NULL) {
        Node **pick = NULL;
        if (a != NULL) pick = &a;
        if (b != NULL && (pick == NULL || b->key < (*pick)->key))
            pick = &b;
        if (c != NULL && (pick == NULL || c->key < (*pick)->key))
            pick = &c;
        Node *p = *pick;
        *pick = p->next;
        *tailNext = p;
        tailNext = &p->next;
    }
    *tailNext = NULL;
    return head;
}
```

令三表结点总数为 N。每个结点取出、接入一次，时间 O(N)；仅用固定个数指针，辅助空间 O(1)。全部为空时返回 NULL。

建议评分（8分）： 正确的三路最小值选择与重连思想2分；完整C实现、空链和尾端处理5分；时间与辅助空间1分。

2. （10分）

基本思想。 第一遍深度优先搜索维护“到当前结点是否已经见过标记”和当前深度；在合格叶结点处只记最长路径的叶指针。第二遍从根找该叶，把沿途结点写入 out。这样不用在每个叶结点复制整条路径。

```
#include <stdbool.h>
#include <stddef.h>
typedef struct Node {
    int data;
    bool mark;
    struct Node *left, *right;
} Node;

static void chooseLeaf(const Node *p, int depth, bool seenMark,
                       const Node **bestLeaf, int *bestLen) {
    if (p == NULL) return;
    ++depth;
    seenMark = seenMark || p->mark;
    if (p->left == NULL && p->right == NULL) {
        if (seenMark && depth > *bestLen) {
            *bestLen = depth;
            *bestLeaf = p;
        }
        return;
    }
    chooseLeaf(p->left, depth, seenMark, bestLeaf, bestLen);
    chooseLeaf(p->right, depth, seenMark, bestLeaf, bestLen);
}

static bool writeToLeaf(const Node *p, const Node *target,
                        const Node *out[], int *len) {
    if (p == NULL) return false;
    out[(*len)++] = p;
    if (p == target) return true;
    if (writeToLeaf(p->left, target, out, len) ||
        writeToLeaf(p->right, target, out, len)) return true;
    --*len;
    return false;
}

bool longestMarkedPath(const Node *root, const Node *out[], int *len) {
    const Node *leaf = NULL;
    int bestLen = 0;
    *len = 0;
    chooseLeaf(root, 0, false, &leaf, &bestLen);
    if (leaf == NULL) return false;
    return writeToLeaf(root, leaf, out, len);
}
```

设结点数为 n、高度为 h。两次 DFS 各至多访问 n 个结点，时间 O(n)；递归栈最多 h 层，额外辅助空间 O(h)，调用者提供的输出数组不计入辅助空间。空树或无标记的根叶路径时 *len=0。

建议评分（10分）： 合格路径与最长目标的思想2分；第一遍选目标叶、第二遍正确输出及失败处理6分；时间与辅助空间2分。

3. （12分）

基本思想。 从 s 深度优先搜索，记录已在当前路径上的顶点。到达 k 条边时，仅当当前点为 t 才计 1；若提前到达 t，则该路径不能再延伸为顶点不重复的 s→t 路径。回溯时撤销访问标记，使不同顶点序列分别计数。

```
#include <stdbool.h>

static int countDfs(const int g[12][12], int V, int u, int t,
                    int usedEdges, int k, bool onPath[12]) {
    if (usedEdges == k) return u == t ? 1 : 0;
    if (u == t) return 0;
    int ans = 0;
    for (int v = 0; v < V; ++v) {
        if (g[u][v] && !onPath[v]) {
            onPath[v] = true;
            ans += countDfs(g, V, v, t, usedEdges + 1, k, onPath);
            onPath[v] = false;
        }
    }
    return ans;
}

int countPaths(const int g[12][12], int V, int s, int t, int k) {
    bool onPath[12] = {false};
    onPath[s] = true;
    return countDfs(g, V, s, t, 0, k, onPath);
}
```

搜索最多扩展 k 条边，每层候选顶点至多 V 个，每个状态扫描矩阵一行至多 V 项，故时间上界为 O(V^(k+1))；访问数组与递归栈合计辅助空间 O(V)。当 s=t,k=0 返回1；s=t,k>0 返回0。题定 V≤12，此范围内计数不超过 int 上限。

建议评分（12分）： 简单路径与恰k边计数思想2分；访问标记、深度边界、回溯及入口等完整C实现8分；时间与辅助空间2分。

2027考研重邮802数据结构模拟试卷
第4套 参考答案

对应第6版题面。程序填空接受等价写法；非唯一的树形、编码或路径按题内规则判分。算法实现仅给出一种正确方法，其他满足题目约束的解法同样有效。

评分建议用于核对作答要点。复杂度须与考生实际实现一致；不能仅因实现与参考代码不同而扣分。

一、程序阅读题

1. （5分）

各空依次为：16；5；6；O(n)；O(n)。

数组最小值是16；对6个元素的递归从 n=6 调用到 n=1，共6层。每层回溯（除基本层）比较一次，故比较5次；递归栈深度为n。

2. （5分）

__1__ = pa；__2__ = free(pa)；__3__ = pb = pb->next；时间复杂度 O(m+n)。

相等时把 A 当前结点接到结果尾部；A 键较小时释放该结点；B 键较小时仅前进 B。每个游标只向前，后续剩余 A 最多再扫描一次。B 原链不修改。

3. （5分）

__1__ = q->count == 4；__2__ = (q->rear + 1) % 4；__3__ = --q->count；单次入队或出队最坏时间复杂度 O(1)。

满队列通过 count==4 判断，入队只移动 rear；出队只移动 front 并减 count，不随队列长度扫描。

4. （5分）

__1__ = t == NULL；__2__ = t->left == NULL && t->right == NULL；__3__ = leafCount(t->left) + leafCount(t->right)；时间复杂度 O(n)。

空树贡献0，叶结点贡献1，其余结点的叶数为左右子树叶数之和；每个实结点只访问一次。样例叶结点为4、5、7，打印 leaves=3。

二、填空题

1. （3分）

依次填 O(n)、O(log n)。递归树有 n 个叶、总基本操作次数 2n-1；最大调用深度为 log_2n+1，因此递归栈占 O(log n)。

2. （3分）

依次填 15、1060。行主序偏移为 2×6+3=15 个元素，首地址 1000+15×4=1060。

3. （3分）

依次填 3、3。先压入1、2、3，弹3、2；再压4弹4、弹1，最后压5弹5。首次弹出前已有3个元素，模拟全过程最大栈高也是3。

4. （3分）

填 0,1,1,2,1,2,3。前 j-1 个字符的最长真前后缀长度（j=2..7）依次为 0,0,1,0,1,2，分别加1；首项固定0。

5. （3分）

依次填 c、原子。Tail(L)=((c,d))，其 Head 是子表 (c,d)，再取 Head 得原子 c。

6. （3分）

填 33。严格二叉Huffman树有L个叶时，总结点数 2L-1；取L=17得33。

7. （3分）

依次填 n、1。强连通要求每个顶点入度、出度至少为1，因此至少n条弧；恰n条时各点入度与出度都恰为1，可由一个覆盖所有顶点的有向环实现。

8. （3分）

填 10。最坏比较次数为 floor(log_2(600))+1=10，亦即 ceil(log_2(600+1))=10。

9. （3分）

填 (1,2,3,5,7,8,4,9)。新键1放8号位，沿路径8→4→2→1与9、5、2逐层交换。

10. （3分）

依次填 16、4。原森林总边数 19-3=16；加入一棵不相连的树，树数从3增至4。

三、综合应用题

1．（10分）

（1）后缀式为 12 4 - 3 5 + * 2 /。

（2）前六个记号依次是 12,4,-,3,5,+。处理后栈底到栈顶为 [8,8]：先有 12-4=8，再有 3+5=8。

（3）结果为 (8×8)÷2=32。

（4）栈最多同时放 3 个整数：读入 5 后栈为 [8,3,5]。

2. （10分）

（1）前四键的插入槽号依次是 3、4、5、6：10→3，17从3探测到4，24从3探测到5，5从5探测到6。

（2）删除17后，4号槽为 **DELETED（墓碑）**，不能改成EMPTY。

（3）查找24从 H(24)=3 开始，依次探测 3→4→5，共 **3次**；墓碑4不使查找提前失败。

（4）31从3开始探测，首个墓碑为4；继续到0号EMPTY后，回用4号。最终槽位：

槽号	0	1	2	3	4	5	6
内容/状态	EMPTY	EMPTY	EMPTY	10	31	24	5

3. （10分）

（1）中序序列：D,B,E,A,F,C,G。

（2）按 D、E、F、G 顺序的左线索：NULL、B、A、C。

（3）按 D、E、F、G 顺序的右线索：B、A、C、NULL。

（4）B 的右指针仍指实际右孩子 E，所以 rtag=0。

核对：叶结点 D/E/F/G 的两孩子指针原本均空，才被改为前驱/后继线索。

4．（10分）

（1）刚确定0并放松其出边后，候选距离（0～4）为 [0,4,9,∞,∞]。

（2）确定1后，2降为6、3为10；再确定2并放松后为 [0,4,6,9,11]。

（3）随后确定3，使4由11降为10。最终距离为 [0,4,6,9,10]，0到4的最短距离 **10**；一条最短路径是 0→1→2→3→4（权值4+2+3+1）。

5．（10分）

（1）按事件0～5，ve=[0,3,2,6,6,8]。

（2）以工期8为汇点的最迟时间，vl=[0,3,2,6,6,8]。

（3）关键活动为 0→1、0→2、1→3、2→4、3→5、4→5；1→4 的时差为2，并非关键活动。

（4）工期 8；全部关键路径是 0→1→3→5 和 0→2→4→5，两条路径的工时均为8。

6. （10分）

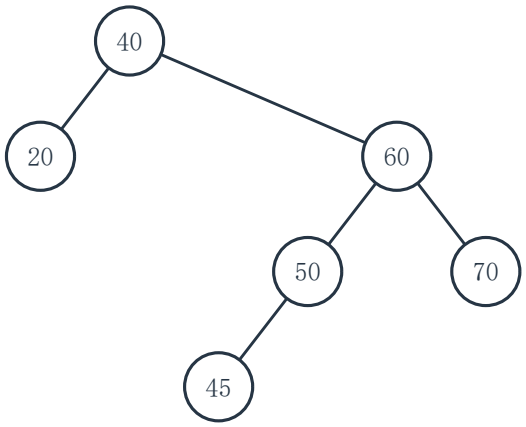
（1）普通 BST：

树形见本题示意图。

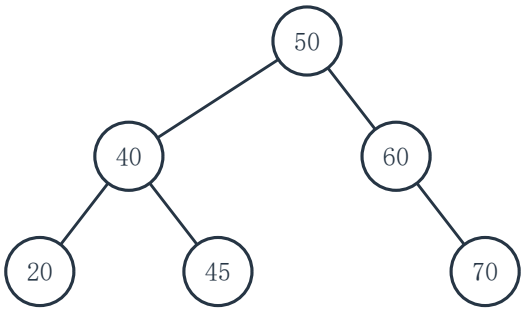
（2）插45后首次失衡的是 40，属于右左（RL）型：先在其右孩子60处右旋，再在40处左旋。AVL 最终为：

树形见本题示意图。

（3）普通 BST 的六键比较次数按输入顺序为 1,2,2,3,3,4，ASL=15/6=2.5；AVL 深度层数为1、2、2、3、3、3，ASL=14/6=7/3≈2.333。



普通BST



最终AVL树

7. （10分）

（1）五次合并依次为 5+8=13、11+13=24、14+19=33、24+33=57、43+57=100。新权值依次 13、24、33、57、100。

（2）WPL=13+24+33+57+100=227 bit（按100个字符统计）。

（3）F 的码长 1；A 的码长 4。

（4）定长编码需 100×3=300 bit，比 Huffman 多 300-227=73 bit。

四、算法设计与分析题

1. （8分）

基本思想。 将环暂时断开，按原 n 个结点依次扫描。区间内的结点立即释放；区间外的原结点按扫描顺序接到新链尾。扫描结束后，若有保留结点就把新尾指回新首并更新尾指针，否则把尾指针设为 NULL。这同时处理原尾删除和全删空。

```
#include <stdlib.h>
typedef struct Node { int key; struct Node *next; } Node;

int deleteRange(Node **tailRef, int n, int a, int b) {
    if (n == 0) return 0;
    Node *cur = (*tailRef)->next;
    (*tailRef)->next = NULL;  /* 暂时展开环 */
    Node *first = NULL, *last = NULL;
    int deleted = 0;
    for (int i = 0; i < n; ++i) {
        Node *next = cur->next;
        if (a <= cur->key && cur->key <= b) {
            free(cur);
            ++deleted;
        } else {
            if (first == NULL) first = cur;
            else last->next = cur;
            last = cur;
        }
        cur = next;
    }
    if (last == NULL) {
        *tailRef = NULL;
    } else {
        last->next = first;
        *tailRef = last;
    }
    return deleted;
}
```

原结点各访问一次，时间 O(n)；固定个数指针，辅助空间 O(1)。n=0 时原尾已为 NULL，直接返回0。

建议评分（8分）： 保留顺序及重建圆环的思想2分；完整C实现、释放与删空/尾结点处理5分；复杂度1分。

2. （10分）

基本思想。 从根沿 BST 查找 x 的分支下行。沿途每个结点都可能成为当前最近键；比较绝对差，差相同时选较小键。遇到 x 可立即停止。空树不改写输出。

```
#include <stdbool.h>
#include <stddef.h>
typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;

bool closestKey(const Node *root, int x, int *out) {
    if (root == NULL) return false;
    bool have = false;
    long long bestDiff = 0;
    int bestKey = 0;
    for (const Node *p = root; p != NULL; ) {
        long long d = (long long)p->key - x;
        if (d < 0) d = -d;
        if (!have || d < bestDiff ||
            (d == bestDiff && p->key < bestKey)) {
            have = true;
            bestDiff = d;
            bestKey = p->key;
        }
        if (p->key == x) break;
        p = x < p->key ? p->left : p->right;
    }
    *out = bestKey;
    return true;
}
```

树高为 h，最多访问一条根到叶的查找路径，时间 O(h)，辅助空间 O(1)；平衡树时 h=O(log n)，最坏退化时 h=O(n)。差值提升为 long long 计算，避免中间减法溢出。

建议评分（10分）： BST分支查找与最近键更新思想2分；完整C实现含空树、并列取小和差值安全6分；复杂度2分。

3. （12分）

基本思想。 失效边从原MST中去掉后，其余树边形成两个连通分量。先从失效边的一个端点出发，仅沿保留的树边标记一个分量。然后扫描原图矩阵，跳过失效边、无边及同分量边，从跨分量边中选最轻的一条；若没有则失效后图不连通。成功时其余 V-2 条树边加这条边构成新MST，并按 原MST总权值-失效边权值+替换边权值 求新权值。并列最轻边任选。

```
#include <stdbool.h>
#include <limits.h>
typedef struct { int u, v, weight; } Edge;

bool repairMST(const int w[100][100], int V,
               const Edge mst[], int failedIndex,
               Edge *replacement, long long *newWeight) {
    bool inFirst[100] = {false};
    int stack[100], top = 0;
    int start = mst[failedIndex].u;
    inFirst[start] = true;
    stack[top++] = start;

    while (top > 0) {
        int u = stack[--top];
        for (int i = 0; i < V - 1; ++i) {
            if (i == failedIndex) continue;
            int v = -1;
            if (mst[i].u == u) v = mst[i].v;
            else if (mst[i].v == u) v = mst[i].u;
            if (v >= 0 && !inFirst[v]) {
                inFirst[v] = true;
                stack[top++] = v;
            }
        }
    }

    int best = INT_MAX;
    Edge chosen = {0, 0, 0};
    int a = mst[failedIndex].u, b = mst[failedIndex].v;
    for (int u = 0; u < V; ++u) {
        for (int v = u + 1; v < V; ++v) {
            if (inFirst[u] == inFirst[v]) continue;
            if ((u == a && v == b) || (u == b && v == a)) continue;
            if (w[u][v] != INT_MAX && w[u][v] < best) {
                best = w[u][v];
                chosen = (Edge){u, v, w[u][v]};
            }
        }
    }
    if (best == INT_MAX) return false;

    long long total = 0;
    for (int i = 0; i < V - 1; ++i) total += mst[i].weight;
    *replacement = chosen;
    *newWeight = total - mst[failedIndex].weight + best;
    return true;
}
```

标记分量时，每弹出一个顶点至多扫描 V-2 条保留树边；再扫描 V×V 矩阵，时间 O(V^2)。布尔数组和栈各至多 V 个元素，辅助空间 O(V)。失败时直到返回都未改写两个输出。V=2 时无可替换边，正确返回 false；零权边合法。

建议评分（12分）： 正确划分失效树边两侧分量的思想2分；分量标记C代码3分；跨分量最轻有效边及断连/并列处理3分；接口输出与总权值2分；复杂度2分。

2027考研重邮802数据结构模拟试卷
第5套 参考答案

对应第6版题面。程序填空接受等价写法；非唯一的树形、编码或路径按题内规则判分。算法实现仅给出一种正确方法，其他满足题目约束的解法同样有效。

评分建议用于核对作答要点。复杂度须与考生实际实现一致；不能仅因实现与参考代码不同而扣分。

一、程序阅读题

1. （5分）

填空：__1__＝24；__2__＝n；__3__＝log_2n；__4__＝n log_2n。

设 F(n) 为返回值，则 F(1)=0，F(n)=2F(n/2)+n。因此 F(2)=2、F(4)=8、F(8)=24。每个非基例层的所有循环合计执行 n 次，共有 log_2n 层，故总时间为 O(n log n)。

2. （5分）

填空：__1__＝pre->next；__2__＝p；__3__＝p->next；__4__＝O(n)；__5__＝O(1)。

删除 p 后，pre 仍指向其前驱，下一待查结点应取 pre->next；保留 p 时，先让 pre 前进到 p，再令 p 前进。每个原数据结点最多处理一次，只使用固定数量指针。

3. （5分）

填空：__1__＝CAP；__2__＝s->top0 + 1 == s->top1；__3__＝s->top1 == CAP；__4__＝O(1)；__5__＝O(1)。

初始化时栈1的栈顶位于数组右边界之外。两个栈顶相邻即无空位；栈1的栈顶退回 CAP 即为空。每次操作仅做固定次数的比较和指针 / 下标修改。

4. （5分）

填空：__1__＝key % m；__2__＝(pos + 1) % m；__3__＝O(m)。

先取题设的初始散列地址；冲突时沿环形地址加 1。for 循环至多检查 m 个槽，所以即使表满，一次插入也会在 O(m) 时间内返回失败。

二、填空题

1.（3分）

填空：__1__＝(2,5,1,4,3,6,7)；__2__＝(2,1,4,3,5,6,7)；__3__＝8。

第一趟中 6 依次越过 2、5、1、4、3，共交换 5 次；第二趟中 5 越过 1、4、3，共交换 3 次。两趟合计 8 次。

2.（3分）

填空：__1__＝a b c d e / - * +。

先把括号内 c-d/e 化为 c d e / -，再与 b 相乘，最后与 a 相加。

3.（3分）

填空：__1__＝m-1；__2__＝m。

甲用一个永不存元素的槽区分空和满；乙有独立的 count，因此 m 个数组槽均可存元素。

4.（3分）

填空：__1__＝(0,1,1,1,2,3,4)。

按题给 1 基定义，j=2、3、4 的前缀均无非空真前后缀，值为 1；j=5、6、7 时相应的最长真前后缀长度依次为 1、2、3，故值为 2、3、4。

5.（3分）

填空：__1__＝10。

两地址相差 (246-200)/2＝23 个元素位置。设每行 c 列，则 A[2,3] 前有 2c+3 个元素，所以 2c+3＝23，c＝10。

6.（3分）

填空：__1__＝18。

任意非空二叉树中，度为 2 的结点数 n_2＝叶结点数 n_0-1＝7。因此总数 n＝n_0+n_1+n_2＝8+3+7＝18。

7.（3分）

填空：__1__＝16。

顶点 5 必须最后。若首项是 0，第二项为 1 时，剩余 2、3、4 可任意排列，有 3!＝6 种；第二项为 2 时，第三项必为 1，随后 3、4 有 2 种，共 8 种。首项为 1 时对称地也是 8 种，合计 16 种。

8.（3分）

填空：__1__＝8、12、14、13；__2__＝4。

首轮中点为 8（值 16），目标 26 在右半；随后中点依次为 12（24）、14（28）、13（26），第 4 次命中。

9.（3分）

填空：__1__＝(2,7,3,10,9,8,5)。

2 先放在下标 7，与父结点下标 3 的 5 交换，再与根结点 3 交换；其他位置不变。

10.（3分）

填空：__1__＝109。

Huffman 编码树是严格二叉树，若叶结点数为 L，则总结点数为 2L-1。故 L＝(217+1)/2＝109。

三、综合应用题

1．（10分）

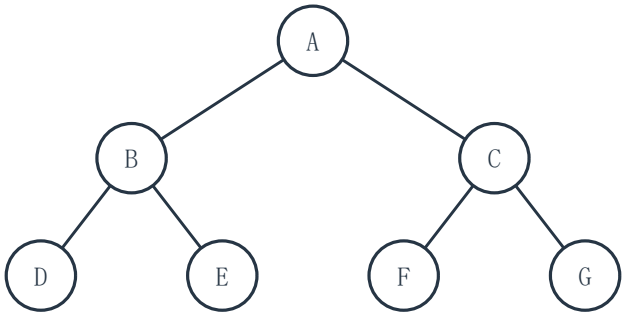
（1）树形：

结点	左孩子	右孩子
A	B	C
B	D	E
C	F	G
D、E、F、G	空	空

先序首项 A 为根；中序在 A 左右分别切成 D、B、E 与 F、C、G，再各由先序确定 B、C 为子树根。

（2）后序：D、E、B、F、G、C、A。

（3）中序前驱 / 后继：D 为“空 / B”；F 为“A / C”。中序线索只写入原空指针，不改变这棵树的孩子关系或中序序列。



重建二叉树

2. （10分）

（1）折半查找判定树：

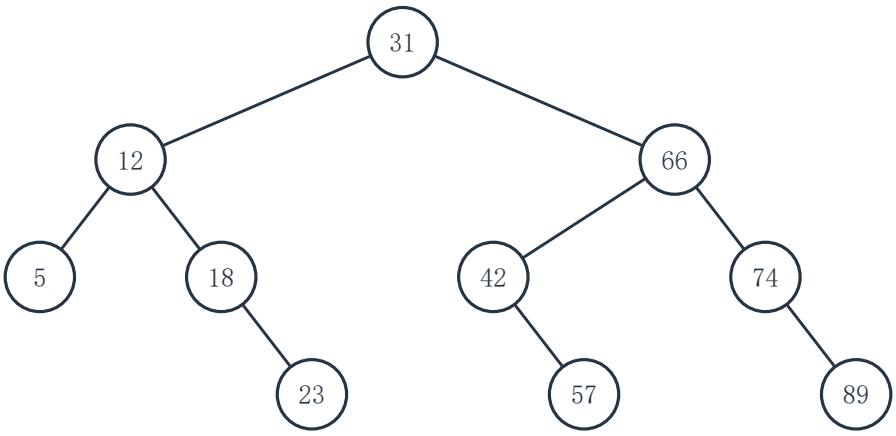
结点	左孩子	右孩子
31	12	66
12	5	18
66	42	74
18	空	23
42	空	57
74	空	89
5、23、57、89	空	空

根取区间 1..10 的下中点 5；各子区间继续按同一规则取下中点。

（2）查找 57：31 → 66 → 42 → 57。

（3）查找 60 失败：31 → 66 → 42 → 57，随后落入空区间。

（4）成功 ASL：各层结点数依次为 1、2、4、3，因此 (1×1+2×2+4×3+3×4)/10＝29/10＝2.9 次。



折半查找判定树

3．（10分）

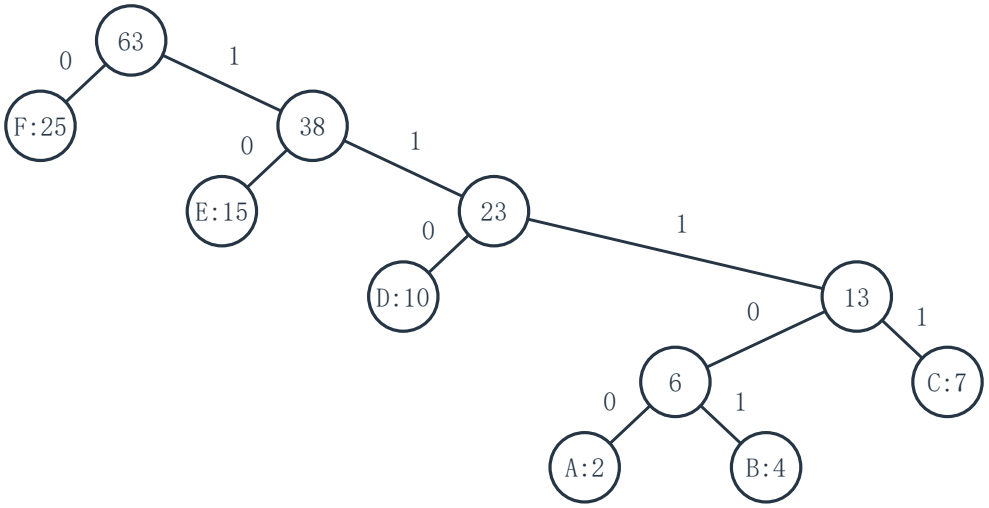
（1）原频率：

步	合并的两棵树	新权
1	2+4	6
2	6+7	13
3	10+13	23
4	15+23	38
5	25+38	63

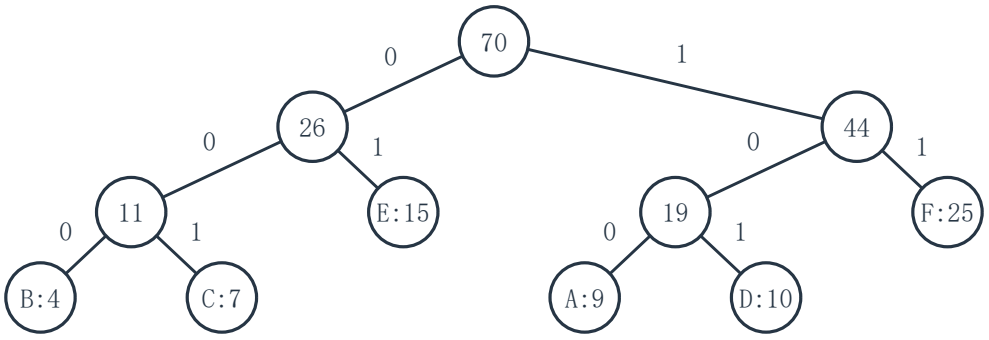
最小 WPL 等于各次新权之和：6+13+23+38+63＝**143**。

（2）只改频率、不改原树：原树中 A 的码长为 5，新频率比原来增加 7，故 WPL＝143+7×5＝**178**。

（3）用新频率重建：此时频率为 4、7、9、10、15、25，依次合并 4+7→11，9+10→19，11+15→26，19+25→44，26+44→70。 新树的最小 WPL＝11+19+26+44+70＝**170**。新树比沿用旧树少 8。



原频率Huffman树（叶结点为字符:频率）



新频率Huffman树（叶结点为字符:频率）

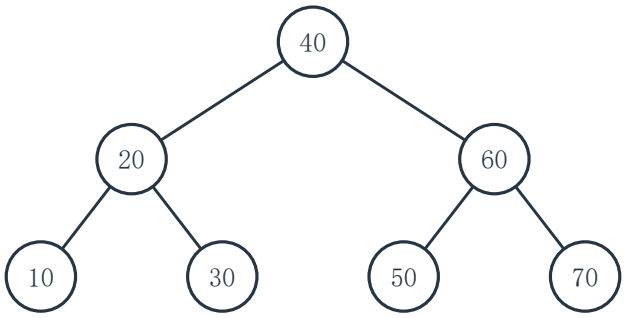
4. （10分）

（1）最终普通二叉排序树：

结点	左孩子	右孩子
40	20	60
20	10	30
60	50	70
10、30、50、70	空	空

（2）后序：10、30、20、50、70、60、40。

（3）查找 55：依次比较 40、60、50，随后应走 50 的空右孩子，故未成功。成功查找时第 1、2、3 层分别有 1、2、4 个结点，ASL＝(1+2×2+4×3)/7＝17/7 次。



普通BST

5. （10分）

（1）插入过程：

关键字	探测地址（按先后）	最终地址	探测次数
22	0	0	1
1	1	1	1
13	2	2	1
11	0、2、4	4	3
24	2、7	7	2
35	2、8	8	2
46	2、9	9	2

最终散列表：

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	22	1	13	空	11	空	空	24	35	46	空

（2）成功 ASL：查找按各自相同的双散列探测序列，次数即上表末列。ASL＝(1+1+1+3+2+2+2)/7＝12/7 次。

6. （10分）

（1）访问序列：DFS：0、1、2、3、4、5、6；BFS：0、1、2、3、4、5、6。两种遍历的访问序列相同，但第一个分量中的树边不同。

（2）生成森林树边：

- DFS：0-1，1-2；3-4，4-5。 - BFS：0-1，0-2；3-4，4-5。

顶点 6 是单点树，无树边。

（3）连通分量为 {0,1,2}、{3,4,5}、{6}，共 **3** 个。每种生成森林均有 7-3＝**4** 条树边。

7. （10分）

（1）先按第二关键字稳定排序：b、e、f、a、c、d（值为 1 的 b、e、f 在前，值为 2 的 a、c、d 在后）。

（2）再按第一关键字稳定排序：b、f、d、e、c、a。此时 A 组内为 b、f、d，B 组内为 e、c，既满足第二关键字次序，也保留完全同键记录 b、f 的原先后。

（3）反向排序的结果：先按第一关键字得到 b、d、f、c、e、a；再按第二关键字稳定排序，最终为 b、f、e、d、c、a。该次序优先按第二关键字分组，不是题设要求的最终主次关键字顺序。

四、算法设计与分析题

1. （8分）

基本思想：用 pre 指向当前连续相等段之前保留的结点，p 指向该段当前结点。只要 p 与下一结点同键，就删除 p，把 pre 直接接到下一结点；直到 p 成为这一段原链的最后一个结点才保留。随后处理下一段。空表直接结束。

C语言实现：

```
#include <stddef.h>
#include <stdlib.h>
typedef struct Node { int key; struct Node *next; } Node;

void keepLastOfEachKey(Node *L) {
    Node *pre = L, *p = L->next;
    while (p != NULL) {
        while (p->next != NULL && p->key == p->next->key) {
            Node *old = p;
            p = p->next;
            pre->next = p;
            free(old);
        }
        pre = p;
        p = p->next;
    }
}
```

复杂度：每个原数据结点最多访问或释放一次，时间 O(n)；除原链及释放操作外只用固定数量指针，辅助空间 O(1)。

建议评分（8分）：说明保留每段最后结点 2 分；正确重连并释放其余结点、处理空表与尾段 4 分；时间 1 分；空间 1 分。

2. （10分）

基本思想：先由出弧表计算各顶点入度。按 P 的顺序模拟 Kahn 拓扑过程：每一步当前顶点必须入度为 0；取走后把它的出邻点入度各减 1。若 k 步均成立，由于原图是 DAG，剩余顶点总能继续取出，故 P 可作某个完整拓扑序的前缀。入度数组是副本，不改 G。

C语言实现：

```
#include <stdbool.h>
#include <stddef.h>
#include <stdlib.h>
typedef struct Arc { int to; struct Arc *next; } Arc;
typedef struct Graph { int V; Arc **head; } Graph;

bool canBePrefix(const Graph *G, const int P[], int k) {
    if (k == 0) return true;
    int *indeg = calloc((size_t)G->V, sizeof *indeg);
    if (indeg == NULL) abort();
    for (int u = 0; u < G->V; ++u)
        for (const Arc *e = G->head[u]; e != NULL; e = e->next)
            ++indeg[e->to];
    for (int i = 0; i < k; ++i) {
        int u = P[i];
        if (indeg[u] != 0) {
            free(indeg);
            return false;
        }
        for (const Arc *e = G->head[u]; e != NULL; e = e->next)
            --indeg[e->to];
    }
    free(indeg);
    return true;
}
```

题设已保证 G 是 DAG，且 P 的元素合法且互异；内存分配失败属于资源错误，上述实现直接终止而不返回错误的真假分类。

复杂度：预处理遍历 V 个顶点和 E 条弧；模拟时每个前缀顶点及其出弧至多再访问一次。时间 O(V+E)，入度副本辅助空间 O(V)。

建议评分（10分）：入度模拟及可扩展性思想 2 分；入度计算、逐项判零与更新、释放内存的代码 6 分；时间 1 分；空间 1 分。

3. （12分）

基本思想：对每个结点递归求左右子树的最大向下高度（以边数计，空子树高度为 -1）。经过该结点的最长路径长度为左高+右高+2；用全局于此次调用的 best 参数记录所有结点的最大值。递归只读原树。

C语言实现：

```
#include <stddef.h>
typedef struct Node { struct Node *left, *right; } Node;

static int heightAndUpdate(const Node *p, int *best) {
    if (p == NULL) return -1;
    int left = heightAndUpdate(p->left, best);
    int right = heightAndUpdate(p->right, best);
    int through = left + right + 2;
    if (through > *best) *best = through;
    return (left > right ? left : right) + 1;
}

int treeDiameter(const Node *root) {
    int best = 0;
    heightAndUpdate(root, &best);
    return best;
}
```

复杂度：每个结点只处理一次，时间 O(n)；递归栈空间 O(h)，h 为树高，最坏退化链时 O(n)。空树和单结点均返回 0。

建议评分（12分）：说明“经过当前结点”与子树内最优同时比较的思想 3 分；空树高度、左右递归、best 更新及返回值等完整代码 7 分；时间 1 分；递归栈空间 1 分。

2027考研重邮802数据结构模拟试卷
第6套 参考答案

对应第6版题面。程序填空接受等价写法；非唯一的树形、编码或路径按题内规则判分。算法实现仅给出一种正确方法，其他满足题目约束的解法同样有效。

评分建议用于核对作答要点。复杂度须与考生实际实现一致；不能仅因实现与参考代码不同而扣分。

一、程序阅读题

1. （5分）

填空：__1__＝65536；__2__＝4；__3__＝O(log log n)；__4__＝O(1)。

x 从 2 开始依次平方为 4、16、256、65536，第四次平方后不再满足 x<n/2。执行 t 次平方后 x＝2^(2^t)，使它达到 n 的常数倍只需 O(log log n) 次；函数仅用固定数量标量变量。

2. （5分）

填空：__1__＝a != NULL || b != NULL；__2__＝a->key <= b->key；__3__＝O(m+n)。

只要任一链表尚有结点就继续。两边均非空且同键时，使用 <= 使本轮先摘 A 的结点；摘出的结点头插到 out，故最终按非递增顺序排列。每轮恰消耗一个原结点。

3. （5分）

填空：__1__＝size(q)；__2__＝p->left == NULL && p->right == NULL；__3__＝O(N)。

进入每层前用 size(q) 固定本层结点数；新入队的孩子留到下一层处理。左右孩子均空才是叶结点。每个结点恰出队一次，队列基本操作按题设为 O(1)。

4. （5分）

填空：__1__＝!empty(q)；__2__＝--indeg[e->to] == 0；__3__＝O(V+E)。

队列非空时取出一个当前入度为零的顶点。删除其每条出弧对应的入度贡献，只有新入度恰为零时才入队。初始化扫描 V 个顶点，随后每个可输出顶点及其出弧至多处理一次。

二、填空题

1. （3分）

填空：__1__＝4。

按题设 front 指队首的前一槽、rear 指队尾，元素数为 (rear-front+7) mod 7＝(2-5+7) mod 7＝4。占用的槽依次是 6、0、1、2。

2. （3分）

填空：__1__＝152。

按行主序，A[2][3] 前有 2×5+3＝13 个 int；首地址＝100+13×4＝152。

3. （3分）

填空：__1__＝3。

首项为 3 时，必须先将 1、2、3 依次入栈并弹出 3。此后完整出栈序列只能是 (3,2,1,4)、(3,2,4,1)、(3,4,2,1)，共 3 种。

4. （3分）

填空：__1__＝0、2；__2__＝7。

合法起点是 0、1、2。起点 0 完整匹配，比较 3 次；起点 1 首字符 b 与 a 失配，比较 1 次；起点 2 完整匹配，比较 3 次。总数 3+1+3＝7。

5. （3分）

填空：__1__＝(c,d)。

Tail(X)＝((c,d),e)，再取第一个元素就是子表 (c,d)，不是单个原子 c。

6. （3分）

填空：__1__＝7。

right 为空的结点是每条兄弟链的末结点。13-7＝6 个非叶结点各有一条孩子链，再加上森林诸根组成的一条根链，故共有 6+1＝7 条兄弟链，也即 7 个空 right。

7. （3分）

填空：__1__＝27。

为使结点数最多，前 4 层先填满，共 1+2+4+8＝15 个结点。第 4 层恰有 2 个叶结点，故其余 6 个结点可各有 2 个第 5 层孩子，最多再有 12 个，总数 15+12＝27。

8. （3分）

填空：__1__＝(1,7,2,9,8,6,4)。

把 1 放入下标 7，先与其父结点下标 3 的 4 交换，再与根下标 1 的 2 交换；最后每个父结点均不大于孩子。

9. （3分）

填空：__1__＝19。

18 个结点共有 36 个孩子指针域；非空指针恰对应树的 17 条边，所以空指针域数＝36-17＝19，也可直接用 n+1。

10. （3分）

填空：__1__＝8。

折半判定树高度最多为 ceil(log_2(180+1))＝8；亦可看作 2^7-1＝127 个关键字不足以形成最坏 7 次查找的容量，而 2^8-1＝255 足够。实际 180 个关键字的下中点规则下存在深度 8 的结点。

三、综合应用题

1．（10分）

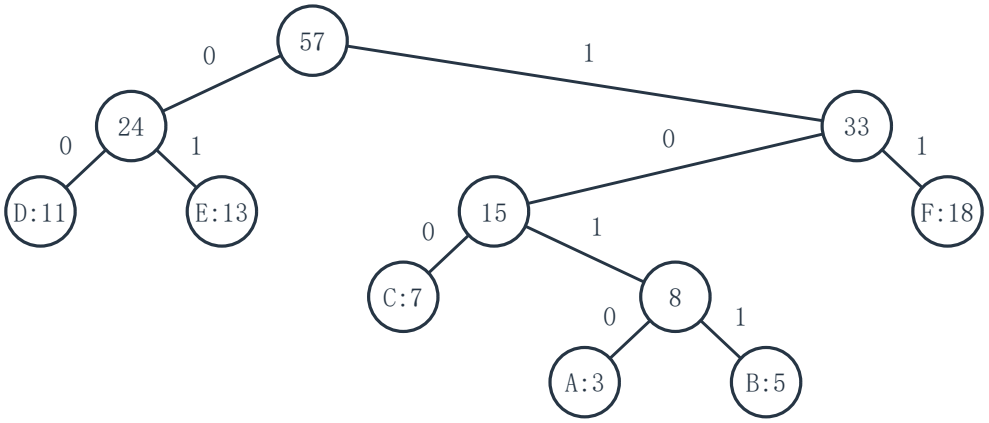
（1）各次合并新权：3+5→8，7+8→15，11+13→24，15+18→33，24+33→57，即 8、15、24、33、57。

（2）一种符合“小权在左、左0右1”的Huffman树：

树形见本题示意图。

故 A、B、C、D、E、F 的编码依次为 1010、1011、100、00、01、11。

（3）WPL：内部合并权之和为 8+15+24+33+57＝137。按叶码长复算为
3×4+5×4+7×3+11×2+13×2+18×2＝137。



Huffman树（叶结点为字符:频率，左0右1）

2. （10分）

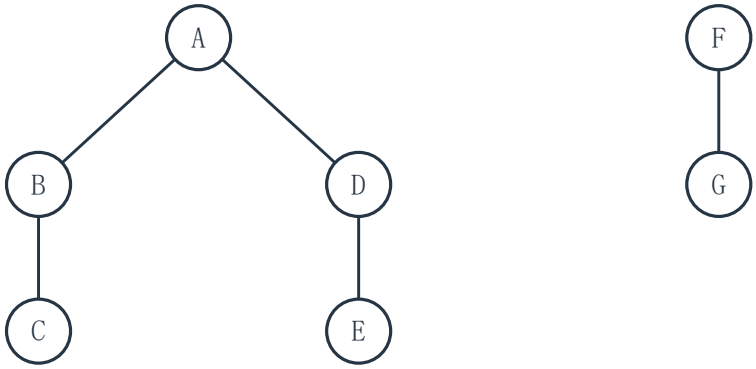
（1）还原的有序森林：两棵树的根依次为 A、F。A 的孩子从左到右为 B、D；B 的孩子为 C；D 的孩子为 E；F 的孩子为 G。

树形见本题示意图。

因为 A.right=F，F 是下一棵树的根；B.right=D，D 是 B 的兄弟而不是 B 的孩子。

（2）逐树后根遍历：C、B、E、D、A、G、F。

（3）最大深度：原森林为 3 层（A→B→C 或 A→D→E）；孩子-兄弟二叉树为 4 层（A→B→D→E，其中 B→D 是 right 边）。



恢复后的有序森林

3．（10分）

（1）访问区间与下标：

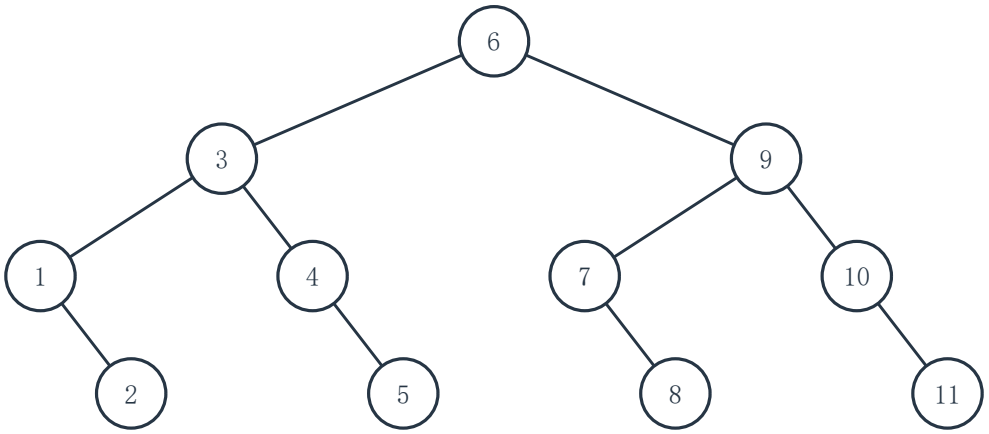
比较	当前闭区间	中点下标	所比关键字	下一步
1	[1,11]	6	25	28>25，转 [7,11]
2	[7,11]	9	33	28<33，转 [7,8]
3	[7,8]	7	28	找到

故 33 在 A[9]，28 在 A[7]；无须推测其他关键字的数值。

（2）完整下标判定树：

下标结点	左孩子	右孩子
6	3	9
3	1	4
9	7	10
1	空	2
4	空	5
7	空	8
10	空	11
2、5、8、11	空	空

（3）成功 ASL：第 1、2、3、4 层分别为 1、2、4、4 个下标；ASL＝(1+2×2+4×3+4×4)/11＝33/11＝3 次。



完整下标判定树

4. （10分）

（1）普通BST：

树形见本题示意图。

各层结点数为 1、2、3、1；成功 ASL＝(1+2×2+3×3+1×4)/7＝18/7。

（2）逐次AVL调整后：

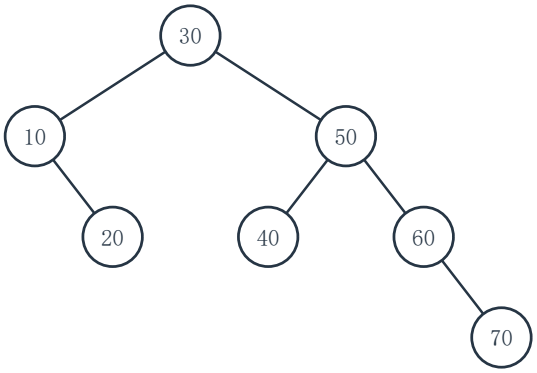
树形见本题示意图。

插入 20 后对 30 作 LR 调整；插入 40 后对右侧 30 作 RL 调整；插入 60 后对 20 作左旋；插入 70 后对 50 作左旋。最终第 1、2、3 层结点数为 1、2、4，成功 ASL＝17/7。

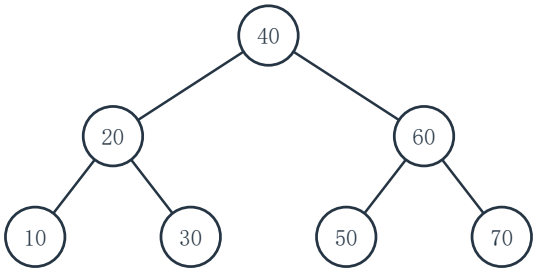
（3）升序数组的折半下标判定树：

下标结点	左孩子	右孩子
4	2	6
2	1	3
6	5	7
1、3、5、7	空	空

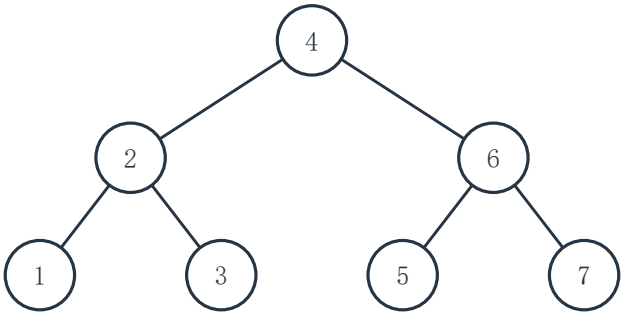
下标 4、2、6 分别对应键 40、20、60。ASL＝(1+2×2+4×3)/7＝17/7。



普通BST



最终AVL树



下标判定树

5．（10分）

（1）事件时刻与总工期：

顶点	0	1	2	3	4	5
ve	0	2	3	6	7	9
vl	0	2	3	6	7	9

正向取前驱“最早时刻+工期”的最大值，逆向取后继“最晚时刻-工期”的最小值。总工期为 **9**。

（2）按题面活动顺序：

活动	0→1	0→2	1→3	2→3	2→4	3→5	4→5	1→4
e	0	0	2	3	3	6	7	2
l	0	0	2	5	3	6	7	6

活动 u→v 的 e＝ve[u]，l＝vl[v]-持续时间。

（3）关键活动：0→1、0→2、1→3、2→4、3→5、4→5。关键路径有 **0→1→3→5** 与 **0→2→4→5**，两者均长 9。

6. （10分）

（1）一棵MST：按权值由小到大取 DE(1)、AB(2)、BC(2)、CD(3)，均不成环且连通五点，**总边权 8**。

（2）从 A 出发的最短距离：

终点	B	C	D	E
距离	2	3	6	7

例如到 D 经 A→C→D 为 3+3＝6，到 E 再经 D→E 为 7。

（3）一棵A源最短路径树：AB(2)、AC(3)、CD(3)、DE(1)，**总边权 9**。它分别实现上表四个最短距离。最小生成树的总权与最短路径树的总权不必相同。

7. （10分）

（1）最终散列表：

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	22	空	35	46	空	空	空	18	29	40	7

（2）逐项探测：

关键字	18	29	40	7	22	35	46
最终地址	7	8	9	10	0	2	3
成功探测次数	1	2	3	4	1	1	2

18、29、40、7 的初始地址均为 7，依次占用 7、8、9、10；35 与 46 的初始地址均为 2。

（3）成功 ASL：(1+2+3+4+1+1+2)/7＝14/7＝2 次。

四、算法设计与分析题

1．（8分）

基本思想：双指针同时扫描两条严格递增链。较小键只在一条链出现，把原结点摘下接到结果尾；相等键在两条链都出现，先保存各自后继再释放两个结点。某条链先结束时，直接把另一条余链接到结果尾。

C语言实现：

```
#include <stddef.h>
#include <stdlib.h>
typedef struct Node { int key; struct Node *next; } Node;

Node *symmetricDiff(Node *A, Node *B) {
    Node dummy = {0, NULL};
    Node *tail = &dummy;
    while (A != NULL && B != NULL) {
        if (A->key < B->key) {
            Node *p = A;
            A = A->next;
            p->next = NULL;
            tail->next = p;
            tail = p;
        } else if (B->key < A->key) {
            Node *p = B;
            B = B->next;
            p->next = NULL;
            tail->next = p;
            tail = p;
        } else {
            Node *oldA = A, *oldB = B;
            A = A->next;
            B = B->next;
            free(oldA);
            free(oldB);
        }
    }
    tail->next = (A != NULL ? A : B);
    return dummy.next;
}
```

dummy 只是栈上的临时哨兵，不是新建的数据结点。结果中的结点均来自原链，输入结点互不共享，因此释放共同键对应的两个结点安全。

复杂度：每轮至少消耗一个原结点，时间 O(m+n)，辅助空间 O(1)。

建议评分（8分）：双指针比较与共键双释放思想 2 分；摘结点接尾、共键释放、余链连接的完整代码 4 分；时间 1 分；空间 1 分。

2. （10分）

基本思想：先序当前段的首元素为根；在线性扫描对应中序段时找到该根，其左侧为左子树、右侧为右子树。按左、右、根递归处理，并用 out 指针顺序写入 post。不创建树结点。

C语言实现：

```
static void fillPost(const int pre[], const int in[],
                     int preLeft, int inLeft, int len,
                     int post[], int *out) {
    if (len == 0) return;
    int root = pre[preLeft];
    int split = inLeft;
    while (in[split] != root) ++split;  /* 题设保证遍历合法且标识互异 */
    int leftSize = split - inLeft;
    fillPost(pre, in, preLeft + 1, inLeft, leftSize, post, out);
    fillPost(pre, in, preLeft + 1 + leftSize, split + 1,
             len - leftSize - 1, post, out);
    post[(*out)++] = root;
}

void toPost(const int pre[], const int in[], int n, int post[]) {
    int out = 0;
    fillPost(pre, in, 0, 0, n, post, &out);
}
```

复杂度：每层为定位根线性扫描中序段，最坏退化树的扫描长度为 n+(n-1)+...+1，时间 O(n^2)；递归栈辅助空间 O(h)，最坏 O(n)。post 是题目提供的输出数组，不计入辅助空间。n=0 时直接返回且不写 post。

建议评分（10分）：先中序划分及左、右、根写入思想 2 分；区间、左右规模、递归偏移、空段处理与输出位置代码 6 分；时间 1 分；空间 1 分。

3. （12分）

基本思想：先扫描既有拓扑序，得到 u、v 的位置。若 u=v，新增自环必成环；若 u 在 v 前面，新弧也符合原序，返回 ORDER_OK。否则原序不再合法：此时只有原图中存在 v 到 u 的有向路径，新增 u→v 才会成环。用只读 DFS / 栈搜索该路径；有路径返回 CYCLE，无路径返回 REORDER。检查过程不改 G，也不输出新序。

C语言实现：

```
#include <stddef.h>
#include <stdlib.h>
typedef struct Arc { int to; struct Arc *next; } Arc;
typedef struct Graph { int V; Arc **head; } Graph;
typedef enum { ORDER_OK, REORDER, CYCLE } State;

State classifyArc(const Graph *G, const int sigma[], int u, int v) {
    if (u == v) return CYCLE;
    int V = G->V;
    int *pos = malloc((size_t)V * sizeof *pos);
    if (pos == NULL) abort();
    for (int i = 0; i < V; ++i) pos[sigma[i]] = i;
    if (pos[u] < pos[v]) {
        free(pos);
        return ORDER_OK;
    }
    free(pos);

    unsigned char *seen = calloc((size_t)V, sizeof *seen);
    int *stack = malloc((size_t)V * sizeof *stack);
    if (seen == NULL || stack == NULL) {
        free(seen);
        free(stack);
        abort();
    }
    int top = 0;
    stack[top++] = v;
    seen[v] = 1;
    while (top != 0) {
        int x = stack[--top];
        if (x == u) {
            free(seen);
            free(stack);
            return CYCLE;
        }
        for (const Arc *e = G->head[x]; e != NULL; e = e->next) {
            int y = e->to;
            if (!seen[y]) {
                seen[y] = 1;
                stack[top++] = y;
            }
        }
    }
    free(seen);
    free(stack);
    return REORDER;
}
```

输入顶点与既有拓扑序均合法；若系统内存不足，程序直接终止，不把资源错误误报成三种图状态之一。

复杂度：位置扫描 O(V)，必要时可达性搜索 O(V+E)，总时间 O(V+E)；位置表、访问标志和栈合计辅助空间 O(V)。

建议评分（12分）：按自环、顺序方向、逆序方向及 v→u 路径作三分类的思想 3 分；位置扫描、可达性搜索、三种返回及释放内存的代码 7 分；时间 1 分；空间 1 分。